

Spring Security com Spring Boot 3

Apostila completa para implementar autenticação e autorização usando o projeto **_SistemaLogin** como base real de aprendizado.

Passo a passo, do pom.xml ao HTML final — cada linha de código explicada.

Spring Boot 3.4.5

Java 17

Spring Security 6

Thymeleaf

H2 / MySQL

Lombok

BCrypt

Sumário

FUNDAMENTOS

1	Introdução ao Spring Security	3
2	Configurando o Projeto	4
3	Arquitetura e Filter Chain	6

IMPLEMENTAÇÃO PASSO A PASSO

4	Passo 1 — Entidade Usuario	8
5	Passo 2 — Repository	10
6	Passo 3 — Classe de Configuração de Segurança	11
7	Passo 4 — UserDetailsService em Detalhes	13
8	Passo 5 — SecurityFilterChain em Detalhes	14
9	Passo 6 — Controller	16
10	Passo 7 — Views com Thymeleaf	18
11	Passo 8 — application.properties	21

REFERÊNCIA

12	Fluxo Completo de Autenticação	22
13	Thymeleaf + Spring Security — Tags sec:	23
14	Boas Práticas e BCrypt	24
15	Erros Comuns e Como Resolver	26
16	Glossário	28

1

Introdução ao Spring Security

Spring Security é o framework de segurança padrão do ecossistema Spring. Ele cuida de **autenticação** (verificar *quem* é o usuário) e **autorização** (verificar o *que* o usuário pode fazer), além de proteção CSRF, gerenciamento de sessão e headers HTTP de segurança.

É adotado amplamente em aplicações corporativas e acadêmicas por ser robusto, altamente configurável e integrado nativamente ao Spring Boot — basta adicionar uma dependência para ele entrar em ação.

💡 Analogia do dia a dia

Pense no Spring Security como o porteiro de um condomínio: ele verifica o crachá (autenticação) e decide quais andares você pode acessar (autorização). Sem configuração nenhuma, ele bloqueia *tudo* por padrão.

Os dois pilares: Autenticação vs Autorização

🔑 Autenticação

- Verifica **quem** é o usuário
- Pergunta: "Você é quem diz ser?"
- Ex: usuário e senha no login
- Acontece **primeiro**
- Resultado: usuário identificado ou acesso negado

🛡️ Autorização

- Verifica **o que** o usuário pode fazer
- Pergunta: "Você tem permissão?"
- Ex: só ADMIN acessa `/admin/**`
- Acontece **depois** da autenticação
- Resultado: acesso permitido ou HTTP 403

O que o Spring Security protege automaticamente?

PILAR 1

🔑 Autenticação

Verifica a identidade via formulário, JWT, OAuth2 ou Basic Auth.

PILAR 2

🛡️ Autorização

Controla o acesso por rota e por role. Ex: só ADMIN acessa `/admin/**`.

PILAR 3

🔒 CSRF

Proteção contra Cross-Site Request Forgery em formulários POST.

PILAR 4**Sessão**

Gerencia sessões ativas, timeout e controle de sessões simultâneas.

PILAR 5**Headers HTTP**

Adiciona headers de segurança: X-Frame-Options, HSTS, CSP e outros.

PILAR 6**Criptografia**

Suporte nativo a BCrypt, PBKDF2, Argon2 para hashing de senhas.

⚠️ Atenção ao adicionar a dependência

Assim que você adicionar `spring-boot-starter-security`, o Spring bloqueia **todas as rotas**

automaticamente — incluindo a raiz `/`. Você precisa configurar o

`SecurityFilterChain` para liberar o que for necessário.

2

Configurando o Projeto

O primeiro passo é criar o projeto no **Spring Initializr** (`start.spring.io`) com as configurações corretas e adicionar todas as dependências necessárias no `pom.xml` .

Configurações no Spring Initializr

Campo	Valor
Project	Maven
Language	Java
Spring Boot	3.4.5
Java	17
Group	<code>com.sistema</code>
Artifact	<code>SistemaLogin</code>
Packaging	Jar

O pom.xml do projeto — cada dependência explicada

```
<!-- Spring Boot 3 exige Java 17 como mínimo -->
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.4.5</version>
</parent>

<properties>
  <java.version>17</java.version>
</properties>

<dependencies>

  <!-- 1. JPA: faz a ponte entre Java e o banco de dados -->
  <dependency>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- 2. SECURITY: o coração desta apostila -->
  <dependency>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>

  <!-- 3. THYMELEAF: motor de templates HTML -->
  <dependency>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>

  <!-- 4. WEB: suporte MVC, HTTP -->
  <dependency>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- 5. THYMELEAF + SECURITY: habilita tags sec: no HTML -->
  <dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
  </dependency>

  <!-- 6. H2: banco em memória para desenvolvimento -->
  <dependency>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>

  <!-- 7. DEVTOOLS: hot reload automático -->
  <dependency>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>
```

```

<!-- 8. LOMBOK: elimina getters/setters/construtores -->
<dependency>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

</dependencies>

```

Dependência	Para que serve	Obrigatória?
<code>starter-security</code>	Núcleo — autentica e autoriza tudo	✅ Sim
<code>starter-data-jpa</code>	Persiste e consulta usuários no banco	✅ Sim
<code>starter-thymeleaf</code>	Renderiza as páginas HTML no servidor	✅ Sim
<code>thymeleaf-extras-springsecurity6</code>	Habilita tags <code>sec:</code> nos templates	✅ Sim
<code>h2</code>	Banco em memória para desenvolvimento	🔄 Dev
<code>lombok</code>	Gera getters, setters e construtores	⚡ Opcional
<code>devtools</code>	Hot reload — reinicia o servidor ao salvar	⚡ Opcional

Spring Boot 3 vs Spring Boot 2

No Spring Boot 2, a configuração de segurança era feita herdando

`WebSecurityConfigurerAdapter`. No Spring Boot 3 essa classe foi **removida**

. A forma correta agora é usar beans com `@Bean` e lambdas — como mostrado nesta apostila.

3

Arquitetura e Filter Chain

O Spring Security funciona como uma **cadeia de filtros** (Security Filter Chain) que intercepta cada requisição HTTP *antes* de ela chegar ao seu Controller. Pense como uma fila de seguranças — cada filtro verifica um aspecto diferente.

Fluxo de uma requisição HTTP

1

Requisição HTTP chega

O navegador faz uma requisição GET ou POST para qualquer rota da aplicação.

2

Security Filter Chain intercepta

Antes de chegar ao Controller, a requisição passa por todos os filtros de segurança registrados pelo Spring.

3

Verificação de autenticação

O filtro verifica se há uma sessão ativa no SecurityContext. Se não houver, redireciona para `/login`.

4

Verificação de autorização

Com base nas regras do `SecurityFilterChain`, verifica se o usuário tem a role necessária para aquela rota.

5

Controller é chamado

Somente se todas as verificações passarem, a requisição chega ao seu Controller e retorna a View.

Os 4 blocos que você vai criar neste projeto

Bloco	Classe/Arquivo	Responsabilidade
Entidade	<code>Usuario.java</code>	Representa o usuário no banco de dados
Repositório	<code>UsuarioRepository.java</code>	Busca o usuário no banco pelo nome
Segurança	<code>ConfiguracaoSeguranca.java</code>	UserDetailsService + SecurityFilterChain
Controller	<code>UsuarioController.java</code>	Trata as requisições e responde com Views

💡 Você não escreve os filtros

Você não precisa implementar nenhum filtro manualmente. O Spring Security já os fornece prontos. Sua única tarefa é

configurar as regras

que esses filtros devem aplicar — isso é feito no `SecurityFilterChain`.

O SecurityContext — guardando o usuário logado

Após a autenticação bem-sucedida, o Spring Security armazena as informações do usuário em um objeto chamado `SecurityContext`. Ele fica disponível em toda a sessão do usuário. É de lá que buscamos dados como o nome do usuário logado usando

`@AuthenticationPrincipal` no Controller.

```
// Como o Spring Security guarda e recupera o usuário logado:

// 1. Após login bem-sucedido (feito internamente pelo Security):
SecurityContextHolder.getContext()
    .setAuthentication(authentication);

// 2. Para recuperar no Controller — forma limpa com anotação:
@GetMapping("/area-logada")
public String areaLogada(@AuthenticationPrincipal User user) {
    String nome = user.getUsername(); // nome do usuário logado
    return "area-logada";
}
```

4

Passo 1 — Entidade Usuario

A entidade é a classe Java mapeada para uma tabela no banco de dados. O Spring Security precisa de pelo menos um campo para identificar o usuário (**username**) e a sua senha (**password**) para funcionar.

A classe Usuario.java do projeto

```
package com.sistema.SistemaLogin.models;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Data;

@Entity           // (1) Marca esta classe como uma tabela no banco
@Data            // (2) Lombok: gera getters, setters, toString, equals, hashCode
public class Usuario {

    @Id            // (3) Chave primária
    @GeneratedValue(strategy = GenerationType.IDENTITY) // (4) Auto-incremento
    private Long id;

    private String nome; // (5) coluna "nome" na tabela – usado como username
    private String senha; // (6) coluna "senha" – DEVE conter hash BCrypt em prod.
}
```

O que cada anotação faz

Anotação	Origem	Efeito
<code>@Entity</code>	JPA (Jakarta)	Cria a tabela <code>usuario</code> no banco automaticamente
<code>@Data</code>	Lombok	Gera todos os getters, setters, <code>equals()</code> , <code>hashCode()</code> e <code>toString()</code>
<code>@Id</code>	JPA (Jakarta)	Define <code>id</code> como chave primária da tabela
<code>@GeneratedValue</code>	JPA (Jakarta)	O banco gera o ID automaticamente: 1, 2, 3...

⚡ O que o Lombok gera para você

Sem o `@Data`, você precisaria escrever manualmente: `getNome()`, `setNome()`, `getSenha()`, `setSenha()`, `getId()`, `setId()`, `toString()`, `equals()` e `hashCode()`. O Lombok elimina todo esse código repetitivo.

Versão expandida sem Lombok (para entender)

```
// Sem Lombok – você escreveria tudo isso:
@Entity
public class Usuario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
    private String senha;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNome() { return nome; }
    public void setNome(String n) { this.nome = n; }
    public String getSenha() { return senha; }
    public void setSenha(String s) { this.senha = s; }
    // ... toString(), equals(), hashCode() ...
}

// Com @Data do Lombok – tudo acima é gerado automaticamente!
```



Role do usuário

O projeto atual armazena apenas `nome` e `senha`. Em sistemas reais, adicione um campo `role` (ex: `"ROLE_USER"` ou `"ROLE_ADMIN"`) para controlar diferentes níveis de acesso. Veja a seção 14 para aprender como.

5

Passo 2 — Repository

O Repository é a interface responsável pelas consultas ao banco de dados. Ao estender `JpaRepository`, você ganha dezenas de métodos prontos (save, findAll, deleteById...) e pode criar queries customizadas apenas pelo nome do método.

UsuarioRepository.java do projeto

```
package com.sistema.SistemaLogin.repositories;

import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import com.sistema.SistemaLogin.models.Usuario;

// JpaRepository<Usuario, Long>:
// - Usuario: a entidade gerenciada por este repositório
// - Long: o tipo do @Id (chave primária)
public interface UsuarioRepository
    extends JpaRepository<Usuario, Long> {

    // Query derivada: Spring cria automaticamente o SQL abaixo
    // SELECT * FROM usuario WHERE nome = ?
    // Optional: proteção contra NullPointerException
    Optional<Usuario> findByNome(String nome);
}
```

Métodos gratuitos do JpaRepository

Método	O que faz
<code>save(entity)</code>	Salva ou atualiza uma entidade no banco
<code>findById(id)</code>	Busca pelo ID, retorna <code>Optional</code>
<code>findAll()</code>	Retorna todos os registros da tabela
<code>deleteById(id)</code>	Remove o registro pelo ID
<code>count()</code>	Conta o total de registros
<code>existsById(id)</code>	Verifica se o registro existe
<code>findByName(nome)</code>	Customizado — query derivada do nome do método

✓ Por que Optional?

`Optional<Usuario>` evita `NullPointerException`. Se o usuário não existir no banco, retorna `Optional.empty()` em vez de `null`. Isso permite usar `.orElseThrow()` para lançar uma exceção com mensagem clara quando o usuário não for encontrado.

Como o Optional funciona na prática

```
// Sem Optional – perigoso!
Usuario u = repositorio.findByNome("joao"); // pode retornar null
u.getNome(); // NullPointerException se u == null!

// Com Optional – seguro e expressivo:

// Opção 1: lançar exceção personalizada
Usuario u = repositorio.findByNome("joao")
    .orElseThrow(() -> new UsernameNotFoundException("Não encontrado"));

// Opção 2: verificar antes de usar
Optional<Usuario> opt = repositorio.findByNome("joao");
if (opt.isPresent()) {
    System.out.println(opt.get().getNome());
}

// Opção 3: valor padrão se não encontrar
Usuario u = repositorio.findByNome("joao")
    .orElse(new Usuario());
```

6

Passo 3 — A Classe de Configuração de Segurança

Esta é a classe mais importante do projeto. Ela centraliza toda a configuração de segurança em um único lugar, usando dois beans essenciais: o `UserDetailsService` e o `SecurityFilterChain`.

Visão geral da classe ConfiguracaoSeguranca.java

```
package com.sistema.SistemaLogin.security;

import java.util.Collections;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.security.web.SecurityFilterChain;
import com.sistema.SistemaLogin.repositories.UsuarioRepository;
import com.sistema.SistemaLogin.models.Usuario;

@Configuration // (A) Marca como classe de configuração Spring
public class ConfiguracaoSeguranca {

    // Bean 1: como o Security carrega o usuário do banco
    @Bean
    public UserDetailsService userDetailsService(UsuarioRepository repo) {
        return username -> { // (B) Lambda: chamado no momento do login
            Usuario usuario = repo.findByNome(username)
                .orElseThrow(() -> new UsernameNotFoundException("Não encontrado"));
            return new User(
                usuario.getNome(),
                usuario.getSenha(),
                Collections.singleton(new SimpleGrantedAuthority("ROLE_USER")));
        };
    }

    // Bean 2: as regras de acesso da aplicação
    @Bean
    public SecurityFilterChain chain(HttpSecurity http) throws Exception {
        return http
            .authorizeHttpRequests(auth -> auth // (C) Regras por rota
                .requestMatchers("/", "/cadastro").permitAll()
                .anyRequest().authenticated())
            .formLogin(login -> login // (D) Login personalizado
                .loginPage("/login")
                .defaultSuccessUrl("/area-logada", true)
                .permitAll())
            .logout(logout -> logout // (E) Configuração do logout
                .logoutSuccessUrl("/login?logout")
                .permitAll())
            .build();
    }
}
```

O que são @Configuration e @Bean?

@Configuration

- Diz ao Spring: "esta classe contém definições de beans"
- Processada durante a inicialização da aplicação
- Substitui o antigo arquivo XML de configuração
- Pode conter múltiplos métodos `@Bean`

@Bean

- O método retorna um objeto gerenciado pelo Spring (IoC)
- O Spring chama o método uma vez e armazena o resultado
- O objeto pode ser injetado em qualquer lugar com `@Autowired`
- Substitui a instanciação manual com `new`

7

Passo 4 — UserDetailsService em Detalhes

O `UserDetailsService` é a **ponte entre o seu banco de dados e o Spring Security**. O framework não sabe como seus dados estão estruturados. Ele chama `loadUserByUsername()` e espera receber um objeto `UserDetails` em troca. Você decide como buscar e montar esse objeto.

```
@Bean
public UserDetailsService userDetailsService(UsuarioRepository repo) {

    // Esta lambda implementa loadUserByUsername(String username)
    // O Spring Security a chama automaticamente no momento do login
    return username -> {

        // PASSO 1: busca o usuário no banco pelo nome digitado no form
        Usuario usuario = repo.findByNome(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "Usuário não encontrado: " + username
            ));

        // Se não achar -> lança a exceção -> login falha automaticamente

        // PASSO 2: constrói e retorna o objeto UserDetails
        // User(username, password, authorities) - classe do Spring Security
        return new User(
            usuario.getNome(),    // username: identificador do usuário
            usuario.getSenha(),  // password: DEVE ser hash BCrypt em produção!
            Collections.singleton(
                new SimpleGrantedAuthority("ROLE_USER") // role do usuário
            )
        );
    };
}
```



Entendendo o lambda

`return username -> { ... }` é uma expressão lambda que implementa a interface funcional `UserDetailsService`. É equivalente a criar uma classe anônima com o método `loadUserByUsername(String username)`. O Java infere isso automaticamente pelo tipo de retorno do método.

O que é SimpleGrantedAuthority?

`GrantedAuthority` representa uma permissão ou papel (role) concedida ao usuário. No Spring Security, roles são prefixadas com `ROLE_`:

Authority	Significado	Como verificar no SecurityConfig
<code>ROLE_USER</code>	Usuário comum	<code>.hasRole("USER")</code>
<code>ROLE_ADMIN</code>	Administrador	<code>.hasRole("ADMIN")</code>
<code>ROLE_MANAGER</code>	Gerente	<code>.hasRole("MANAGER")</code>

⚠ `hasRole()` vs `hasAuthority()`

`hasRole("USER")` adiciona o prefixo `ROLE_` automaticamente → compara com `ROLE_USER`.

`hasAuthority("ROLE_USER")` compara o valor exato → você precisa incluir o prefixo `ROLE_` manualmente.

8

Passo 5 — SecurityFilterChain em Detalhes

O `SecurityFilterChain` é onde você define as regras de segurança: quem pode acessar o quê, como funciona o login e o logout. Ele substitui completamente o antigo

`WebSecurityConfigurerAdapter`.

Bloco 1 — authorizeHttpRequests: quem acessa o quê

```
.authorizeHttpRequests(auth -> auth

    // Qualquer pessoa (mesmo sem login) acessa "/" e "/cadastro"
    .requestMatchers("/", "/cadastro").permitAll()

    // Todas as outras rotas exigem login
    .anyRequest().authenticated()

)
```

Outras opções úteis de autorização

```
// Apenas para ADMIN:
.requestMatchers("/admin/**").hasRole("ADMIN")

// Para ADMIN ou MANAGER:
.requestMatchers("/relatorios/**").hasAnyRole("ADMIN", "MANAGER")

// Libera arquivos estáticos (CSS, JS, imagens):
.requestMatchers("/css/**", "/js/**", "/images/**").permitAll()

// Permite apenas usuários autenticados (qualquer role):
.requestMatchers("/perfil").authenticated()
```

⚠ Ordem das regras importa!

As regras são avaliadas

na ordem em que foram declaradas

. Sempre coloque as regras mais específicas antes das mais genéricas. O

`.anyRequest().authenticated()` deve ser

sempre o último

.

Bloco 2 — formLogin: configurando o login personalizado

```
.formLogin(login -> login

    // Nossa página de login personalizada (em vez da gerada pelo Spring)
    .loginPage("/login")

    // Após login bem-sucedido, redireciona SEMPRE para /area-logada
    // 0 "true" força esse destino mesmo que o usuário tentasse outra página
    .defaultSuccessUrl("/area-logada", true)

    // Permite acesso à página de login sem autenticação
    .permitAll()

)
```

💡 Sem loginPage()?

Se você não configurar `loginPage("/login")`, o Spring Security gera automaticamente uma página de login genérica. Útil para desenvolvimento rápido, mas em produção use sempre uma página personalizada.

Bloco 3 — logout: configurando o logout

```
.logout(logout -> logout

    // Após logout, redireciona para /login?logout
    // 0 parâmetro ?logout pode ser detectado no template Thymeleaf
    // para exibir uma mensagem: "Você saiu com sucesso!"
    .logoutSuccessUrl("/login?logout")

    .permitAll()

)
```

🚫 Logout deve ser POST, não GET

O Spring Security exige POST para logout como proteção CSRF. Um link `<a href="/logout">` faz GET e pode não funcionar corretamente. Use sempre um formulário: `<form th:action="@{/logout}" method="post">`

9

Passo 6 — Controller

O Controller é o intermediário entre o usuário e os dados. Ele recebe requisições HTTP, processa a lógica necessária e retorna o nome do template que o Thymeleaf vai renderizar.

UsuarioController.java do projeto

```
package com.sistema.SistemaLogin.controllers;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.core.userdetails.User;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller // Marca como Controller MVC – retorna nomes de templates
public class UsuarioController {

    @Autowired
    private UsuarioRepository usuarioRepository; // injeção automática

    // (A) Página inicial – rota pública
    @GetMapping("/")
    public String index() {
        return "index"; // → templates/index.html
    }

    // (B) Exibe o formulário de cadastro
    @GetMapping("/cadastro")
    public String formCadastro(Model model) {
        model.addAttribute("usuario", new Usuario()); // objeto vazio para o form
        return "cadastro"; // → templates/cadastro.html
    }

    // (C) Processa o formulário de cadastro
    @PostMapping("/cadastro")
    public String cadastrarUsuario(@ModelAttribute Usuario usuario) {
        usuarioRepository.save(usuario); // persiste no banco
        return "redirect:/login"; // padrão PRG – evita resubmissão
    }

    // (D) Área protegida – só usuários logados chegam aqui
    @GetMapping("/area-logada")
    public String areaLogada(Model model, @AuthenticationPrincipal User user) {
        model.addAttribute("usuarioLogado", user.getUsername()); // passa nome ao template
        return "area-logada"; // → templates/area-logada.html
    }
}
```


Analizando cada elemento

Elemento	O que faz
<code>@Controller</code>	Retorna nomes de templates (diferente de <code>@RestController</code> que retorna JSON)
<code>@Autowired</code>	O Spring injeta o Repository automaticamente — sem <code>new UsuarioRepository()</code>
<code>@GetMapping</code>	Responde a requisições HTTP GET na rota especificada
<code>@PostMapping</code>	Responde a requisições HTTP POST (envio de formulário)
<code>Model</code>	Mapa chave→valor para passar dados do Controller ao template Thymeleaf
<code>@ModelAttribute</code>	Preenche automaticamente o objeto <code>Usuario</code> com os dados do formulário
<code>@AuthenticationPrincipal</code>	Injeta o objeto <code>User</code> do Spring Security com os dados do usuário logado
<code>redirect:/login</code>	Redireciona o navegador (padrão PRG — evita duplo envio do formulário)

Por que o login não tem método POST no Controller?

O Spring Security intercepta o `POST /login`

antes

de qualquer Controller. Você nunca precisa criar um `@PostMapping("/login")` — o framework processa as credenciais automaticamente e redireciona conforme configurado.

10

Passo 7 — Views com Thymeleaf

O Thymeleaf é o motor de templates que processa os arquivos HTML no servidor, substituindo expressões especiais pelos dados reais antes de enviar ao navegador. Todos os atributos começam com `th:`.

index.html — Página inicial pública

```
<!DOCTYPE html>
<html lang="pt-br">
<body>
  <h2>Bem-vindo ao sistema!</h2>
  <a href="/cadastro">Cadastrar-se</a>
  <a href="/login">Login</a>
</body>
</html>
```

Página simples, HTML puro. Acessível por todos porque configuramos

```
.requestMatchers("/").permitAll() .
```

login.html — Campos obrigatórios do Spring Security

⚠ Regra crítica — nomes de campos

Os campos do formulário de login

devem

obrigatoriamente se chamar `name="username"` e `name="password"`. Esses são os nomes que o Spring Security espera por padrão. Se usar outros nomes, o login nunca vai funcionar.

```
<!DOCTYPE html>
<html lang="pt-br">
<body>
  <!-- O Spring Security intercepta este POST – sem método no Controller -->
  <form action="/login" method="post">

    <label for="nome">Nome:</label>
    <!-- name="username" é obrigatório! O Spring Security espera este nome -->
    <input type="text" id="nome" name="username" required/>

    <label for="senha">Senha:</label>
    <!-- name="password" é obrigatório! O Spring Security espera este nome -->
    <input type="password" id="senha" name="password" required/>

    <button type="submit">Entrar</button>
  </form>
</body>
</html>
```

💡 Quer usar nomes diferentes?

Configure no `SecurityFilterChain` :

```
.formLogin(l -> l.usernameParameter("nome").passwordParameter("senha"))
```

cadastro.html — Formulário com Thymeleaf

```
<!DOCTYPE html>
<!-- xmlns importa o namespace do Thymeleaf -->
<html lang="pt-br" xmlns="https://www.thymeleaf.org">
<body>
  <!--
    th:action="@{/cadastro}": URL Expression – gera a URL correta
    considerando o context path da aplicação.
    th:object="${usuario}": vincula o objeto "usuario" do Model
    ao formulário. Campos usam *{campo} para referenciar atributos.
  -->
  <form th:action="@{/cadastro}" th:object="${usuario}" method="post">

    <label>Nome:</label>
    <!--
      th:field="*{nome}" é equivalente a:
      name="nome" id="nome" value="${usuario.nome}"
      0 asterisco (*) referencia o objeto do th:object
    -->
    <input th:field="*{nome}" type="text" placeholder="nome..." required/>

    <label>Senha:</label>
    <input th:field="*{senha}" type="password" placeholder="senha..." required/>

    <button type="submit">Cadastrar</button>
  </form>
</body>
</html>
```

Expressão Thymeleaf	Tipo	O que faz
<code>@{/cadastro}</code>	URL Expression	Gera a URL considerando o context path da aplicação
<code>\${usuario}</code>	Variable Expression	Acessa o atributo "usuario" do Model
<code>*{nome}</code>	Selection Expression	Acessa o campo "nome" do objeto vinculado no <code>th:object</code>
<code>th:field</code>	Atributo	Gera automaticamente <code>name</code> , <code>id</code> e <code>value</code>

area-logada.html — Área protegida com dados do usuário

```
<!DOCTYPE html>
<html lang="pt-br" xmlns="https://www.thymeleaf.org">
<body>
  <!--
    th:text substitui o conteúdo do elemento pelo valor da expressão.
    Strings literais ficam entre aspas simples dentro de ${...}.
    ${usuarioLogado} acessa o atributo do Model adicionado pelo Controller:
        model.addAttribute("usuarioLogado", user.getUsername());
  -->
  <h2 th:text="'Usuário ' + ${usuarioLogado} + ' está logado!'"></h2>

  <!-- Link de logout (idealmente use form POST em produção) -->
  <a href="/logout">Sair</a>
</body>
</html>
```

11

Passo 8 — application.properties

O arquivo `application.properties` configura o comportamento do Spring Boot. O projeto usa H2 em memória para desenvolvimento, mas pode ser migrado para MySQL em produção.

Configuração atual — H2 em memória

```
# — Banco H2 em memória (dados perdidos ao reiniciar) —
spring.datasource.url=jdbc:h2:mem:clinicadb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Habilita o console web do H2 em /h2-console
spring.h2.console.enabled=true
```

Configuração para MySQL (produção)

```
# — Conexão MySQL —
spring.datasource.url=jdbc:mysql://localhost:3306/sistema_login?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=sua_senha_aqui
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# — JPA / Hibernate —
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# — Thymeleaf —
spring.thymeleaf.cache=false
```

Valor de <code>ddl-auto</code>	Comportamento	Quando usar
<code>update</code>	Cria/atualiza tabelas sem apagar dados	Desenvolvimento
<code>create</code>	Recria as tabelas do zero a cada restart	Testes locais
<code>validate</code>	Valida o schema sem alterar nada	Produção (seguro)
<code>none</code>	Sem nenhuma ação do Hibernate	Produção

 H2 em memória não persiste dados

O banco H2 em memória (`mem:`) é

apagado

quando a aplicação para. Todos os usuários cadastrados somem. Use apenas para desenvolvimento e testes. Em produção, use sempre MySQL, PostgreSQL ou outro banco persistente.

12

Fluxo Completo de Autenticação

O que acontece internamente entre o clique em "Entrar" e o redirecionamento para `/area-logada`. Entender este fluxo é essencial para depurar problemas de login.

1

Usuário submete o formulário

`POST /login` com campos `username` e `password` — interceptado pelo Spring Security **antes** de qualquer Controller.

2

UsernamePasswordAuthenticationFilter

Filtro interno do Spring que extrai as credenciais do request e cria um token de autenticação para processar.

3

UserDetailsService é chamado

O framework chama `loadUserByUsername(username)` — nosso lambda busca o usuário no banco via `UsuarioRepository.findByNome()`.

4

Comparação de senha

O Spring compara a senha digitada com a armazenada no banco. **Em produção:** usa BCrypt para comparar o hash. Se as senhas não conferem, redireciona para `/login?error`.

5

SecurityContext atualizado

O usuário fica "logado" na sessão HTTP.

`SecurityContextHolder.setAuthentication()` é chamado internamente — o usuário agora é o "Principal".

6

Redirect para /area-logada

Configurado em `defaultSuccessUrl("/area-logada", true)`. O Controller recebe a requisição e `@AuthenticationPrincipal` injeta o usuário logado.

Fluxo de acesso não autorizado

1

Usuário sem login tenta acessar /area-logada

Acesso direto à URL protegida.

2

Security Filter Chain intercepta

Verifica o SecurityContext — nenhum usuário autenticado encontrado.

3

Redirecionamento automático para /login

O Spring Security redireciona automaticamente. Você não precisa escrever nenhum código para isso.

13

Thymeleaf + Spring Security — Tags sec:

A dependência `thymeleaf-extras-springsecurity6` adiciona atributos especiais ao Thymeleaf para mostrar/ocultar elementos HTML baseado nas permissões do usuário logado — sem nenhuma lógica no Controller.

Configurando o namespace sec:

```
<html
  xmlns:th="http://www.thymeleaf.org"
  xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
```

sec:authorize — exibindo elementos por permissão

```
<!-- Apenas para visitantes NÃO logados -->
<div sec:authorize="isAnonymous()">
  <a href="/login">Fazer Login</a>
  <a href="/cadastro">Cadastrar</a>
</div>

<!-- Apenas para usuários LOGADOS -->
<div sec:authorize="isAuthenticated()">
  Olá, <strong sec:authentication="name"></strong>!
</div>

<!-- Apenas para ADMINS -->
<div sec:authorize="hasRole('ADMIN')">
  <a href="/admin">Painel Administrativo</a>
</div>

<!-- Para USER ou ADMIN -->
<div sec:authorize="hasAnyRole('USER', 'ADMIN')">
  <a href="/area-logada">Minha Área</a>
</div>

<!-- Logout correto: formulário POST -->
<form sec:authorize="isAuthenticated()"
  th:action="@{/logout}" method="post">
  <button type="submit">Sair</button>
</form>
```

sec:authentication — exibindo dados do usuário logado

```
<!-- Exibe o nome do usuário logado -->
<span sec:authentication="name"></span>

<!-- Exibe as roles do usuário -->
<span sec:authentication="principal.authorities"></span>

<!-- Detectar parâmetros de URL (logout e erro de login) -->
<div th:if="${param.logout}" style="color:green">
    Você saiu com sucesso!
</div>
<div th:if="${param.error}" style="color:red">
    Usuário ou senha inválidos!
</div>
```

Expressão sec:	O que verifica
<code>isAuthenticated()</code>	Usuário está logado (qualquer role)
<code>isAnonymous()</code>	Usuário não está logado
<code>hasRole('ADMIN')</code>	Usuário tem exatamente a role ADMIN
<code>hasAnyRole('A','B')</code>	Usuário tem role A ou B
<code>sec:authentication="name"</code>	Exibe o username do usuário logado

14

Boas Práticas e BCrypt

O projeto atual salva a senha em **texto puro** no banco. Isso é uma falha grave de segurança. Veja como corrigir adicionando criptografia BCrypt.

NUNCA salve senhas em texto puro em produção

Se o banco for comprometido (SQL Injection, backup exposto, acesso indevido), todas as senhas estarão visíveis. Isso viola a LGPD e pode resultar em responsabilidade legal.

Por que BCrypt?

VANTAGEM 1

Salt automático

Gera um valor aleatório (salt) diferente a cada encode, tornando ataques de rainbow table impossíveis.

VANTAGEM 2

Lento por design

O custo computacional dificulta ataques de força bruta. Você controla o "work factor".

VANTAGEM 3

Hash diferente

A mesma senha sempre gera hashes diferentes. Nunca compare hashes diretamente.

VANTAGEM 4

Padrão do Spring

Recomendado e integrado nativamente. Sem dependências extras.

Como adicionar BCrypt ao projeto

Passo 1 — Registrar o Bean na ConfiguracaoSeguranca

```
// Adicione este método na classe ConfiguracaoSeguranca:
@Bean
public BCryptPasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
    // BCryptPasswordEncoder() usa strength=10 por padrão
    // Você pode customizar: new BCryptPasswordEncoder(12)
}
```

Passo 2 — Criptografar a senha ao cadastrar

```
// No UsuarioController, adicione o encoder:
@Autowired
private BCryptPasswordEncoder passwordEncoder;

@PostMapping("/cadastro")
public String cadastrarUsuario(@ModelAttribute Usuario usuario) {
    // Criptografa ANTES de salvar no banco
    String hash = passwordEncoder.encode(usuario.getSenha());
    usuario.setSenha(hash);
    usuarioRepository.save(usuario);
    return "redirect:/login";
}

// O hash BCrypt sempre começa com $2a$10$ e tem 60 caracteres:
// $2a$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWly
```

Passo 3 — Adicionar múltiplas Roles

```
// Adicione o campo role na entidade Usuario:
@Entity @Data
public class Usuario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
    private String senha;
    private String role; // ex: "ROLE_USER" ou "ROLE_ADMIN"
}

// No UserDetailsService, use a role do banco:
return new User(
    usuario.getNome(),
    usuario.getSenha(),
    Collections.singleton(new SimpleGrantedAuthority(usuario.getRole()))
);
```

15

Erros Comuns e Como Resolver

Os erros mais frequentes ao configurar Spring Security pela primeira vez — e como identificar e corrigir cada um.

✗ ERRO 1

There is no PasswordEncoder mapped for the id "null"

Causa: A senha foi salva sem encodar, ou o bean `PasswordEncoder` não foi configurado. O Spring tenta identificar o tipo de encoder pelo prefixo da senha (ex: `{bcrypt}...`).

Solução: Adicione o bean `BCryptPasswordEncoder` na classe de configuração E use `passwordEncoder.encode(senha)` ao salvar o usuário.

✗ ERRO 2

Login sempre retorna erro mesmo com credenciais corretas

Causas mais comuns:

1. Campo do form não se chama `name="username"` / `name="password"`
2. Senha no banco está em texto puro mas o encoder está configurado (ou vice-versa)
3. O `loadUserByUsername` está buscando pelo campo errado (email vs nome)

Como verificar: A senha BCrypt sempre começa com `$2a$`. Se no banco estiver texto puro, o problema está no cadastro.

✗ ERRO 3

403 Forbidden em CSS, JS e imagens

Causa: O Spring Security está bloqueando os arquivos estáticos.

Solução: Adicione no `SecurityFilterChain` :

```
.requestMatchers("/css/**", "/js/**", "/images/**",  
"/webjars/**").permitAll()
```

✖ ERRO 4**Logout via link GET retorna 403 ou não funciona**

Causa: O Spring Security exige POST para logout como proteção CSRF. Um `<a href="/logout">` faz GET.

Solução: Use sempre um formulário:

```
<form th:action="@{/logout}" method="post"><button>Sair</button></form>
```

✖ ERRO 5**WebSecurityConfigurerAdapter — cannot find symbol**

Causa: Você está seguindo um tutorial do Spring Boot 2.x. Essa classe foi completamente removida no Spring Boot 3.

Solução: Use o estilo moderno com `SecurityFilterChain` como `@Bean`, conforme mostrado nesta apostila.

✖ ERRO 6**H2 Console retorna 403**

Causa: O Spring Security bloqueia o console H2 por padrão (proteção de frame + CSRF).

Solução: Adicione na configuração:

```
.requestMatchers("/h2-console/**").permitAll() e também .headers(h -> h.frameOptions(f -> f.disable()))
```


✓ Resumo das classes e arquivos do projeto

- **models/Usuario.java**
 - Entidade JPA com nome e senha
- **repositories/UsuarioRepository.java**
 - Interface com `findByNome()`
- **security/ConfiguracaoSeguranca.java**
 - UserDetailsService + SecurityFilterChain
- **controllers/UsuarioController.java**
 - GET/POST /cadastro, GET /area-logada
- **templates/index.html**
 - Página inicial pública
- **templates/login.html**
 - Form com campos `username` e `password`
- **templates/cadastro.html**
 - Form com `th:object` e `th:field`
- **templates/area-logada.html**
 - Área protegida com `th:text` e `${usuarioLogado}`
- **resources/application.properties**
 - Configuração H2/MySQL

16

Glossário

Termos fundamentais do Spring Security que você encontrará em documentações, fóruns e projetos reais.

Termo	Definição
<code>Authentication</code>	Processo de verificar a identidade do usuário. Resultado: usuário identificado (Principal) ou acesso negado.
<code>Authorization</code>	Processo de verificar se o usuário autenticado tem permissão para acessar um recurso específico.
<code>Principal</code>	O usuário atualmente autenticado. Representado pelo objeto <code>UserDetails</code> armazenado no <code>SecurityContext</code> .
<code>GrantedAuthority</code>	Representa uma permissão ou role concedida ao usuário. Ex: <code>ROLE_USER</code> , <code>ROLE_ADMIN</code> .
<code>UserDetails</code>	Interface do Spring Security que representa o usuário: username, password, authorities, enabled, etc.
<code>UserDetailsService</code>	Interface com <code>loadByUsername()</code> . Implementada para buscar o usuário do banco no momento do login.
<code>SecurityContext</code>	Objeto que armazena o <code>Authentication</code> (usuário logado) durante a sessão. Acessado via <code>SecurityContextHolder</code> .
<code>SecurityFilterChain</code>	Cadeia de filtros que definem as regras de segurança. Substitui o antigo <code>WebSecurityConfigurerAdapter</code> .
<code>BCrypt</code>	Algoritmo de hash para senhas. Lento por design (dificulta brute force) e inclui salt automático.
<code>CSRF</code>	Cross-Site Request Forgery — ataque onde site malicioso engana o browser a fazer requisições não autorizadas. Spring Security habilita proteção por padrão.
<code>Session</code>	Mecanismo HTTP que mantém o estado do usuário entre requisições. O Spring Security cria uma sessão após login bem-sucedido.
<code>@AuthenticationPrincipal</code>	Anotação que injeta o usuário logado (<code>UserDetails</code>) diretamente como parâmetro de método no Controller.

`permitAll()`

Permite o acesso de qualquer pessoa (autenticada ou não) à rota configurada.

`authenticated()`

Exige que o usuário esteja autenticado (logado) para acessar a rota.

`sec:authorize`

Atributo Thymeleaf para mostrar/ocultar elementos HTML baseado em permissões do usuário.

`sec:authentication`

Atributo Thymeleaf para exibir informações do usuário autenticado (nome, roles, etc.).

`th:action`

Define a URL de envio de um formulário no Thymeleaf, gerenciando o context path automaticamente.

`th:field`

Associa um campo de formulário ao atributo de um objeto, gerando `name`, `id` e `value` automaticamente.

`Model`

Mapa chave→valor usado pelo Controller para passar dados aos templates Thymeleaf.

Apostila — Spring Security com Spring Boot 3 | Projeto: _SistemaLogin
Material didático para fins acadêmicos. Baseado no projeto de William Firmino.